Task 1:

```
protocol Payable {
   var name: String { get }
   func monthlyPay() -> Double
}
extension Payable {
   func payslip() -> String {
      return "\(name): Monthly Pay = ₹\(monthlyPay())"
   }
}
struct FullTimeEmployee: Payable {
   let name: String
   let annualSalary: Double
   func monthlyPay() -> Double {
      return annualSalary / 12
   }
}
struct Intern: Payable {
   let name: String
   let stipend: Double
   func monthlyPay() -> Double {
      return stipend
   }
}
struct Contractor: Payable {
   let name: String
```

```
    let hourlyRate: Double
    let hours: Double
    func monthlyPay() -> Double {
        return hourlyRate * hours
    }
}
let employees: [any Payable] = [
    FullTimeEmployee(name: "Asha", annualSalary: 600000),
    Intern(name: "Ravi", stipend: 15000),
    Contractor(name: "Meena", hourlyRate: 500, hours: 80)
]
for employee in employees {
    print(employee.payslip())
}
```

Output:

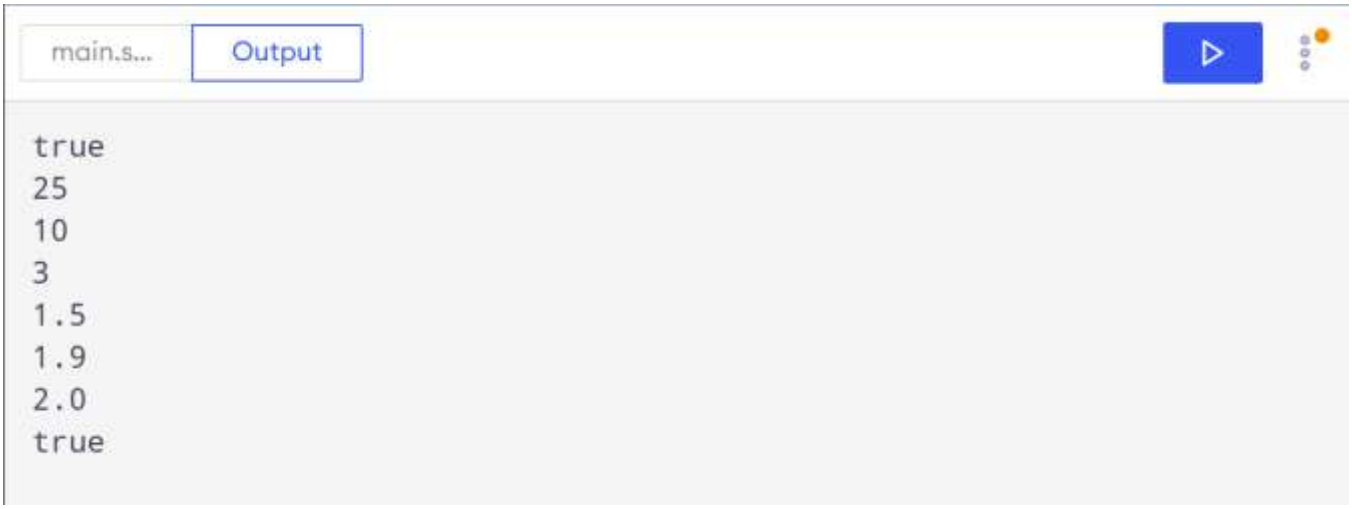
Task 2:

```
extension Int {
    var isEven: Bool {
        return self % 2 == 0
    }
```

```
    func squared() -> Int {
        return self * self
    }
}
print(4.isEven)
print(5.squared())
extension Array where Element == Int {
    var sum: Int {
        return reduce(0, +)
    }
}
print([1, 2, 3, 4].sum)
extension String {
    var wordCount: Int {
        return split { $0.isWhitespace }.count
    }
}
print("Hello Swift World".wordCount)
struct Version: Comparable {
    let major: Int
    let minor: Int
    static func < (lhs: Version, rhs: Version) -> Bool {
        if lhs.major != rhs.major {
            return lhs.major < rhs.major
        }
        return lhs.minor < rhs.minor
```

```
    }
}
let versions = [
    Version(major: 2, minor: 0),
    Version(major: 1, minor: 9),
    Version(major: 1, minor: 5)
]
let sortedVersions = versions.sorted {
    $0 < $1
}
for version in sortedVersions {
    print("\(version.major).\(version.minor)")
}
print(Version(major: 1, minor: 9) < Version(major: 2, minor: 0))
```

Output:



Task 3:

```
protocol Flyable {
   func fly()
}
protocol Swimmable {
   func swim()
}
struct Duck: Flyable, Swimmable {
   func fly() {
      print("Duck is flying")
   }
   func swim() {
      print("Duck is swimming")
   }
}
struct Eagle: Flyable {
   func fly() {
      print("Eagle is flying")
   }
}
struct Fish: Swimmable {
   func swim() {
      print("Fish is swimming")
   }
}
func showOff(_ animal: Flyable & Swimmable) {
```

```
    animal.fly()
    animal.swim()
}
let duck = Duck()
showOff(duck)
```

Output: